

Lecture 18: Binary Arithmetic Using 2's Complement (Signed Addition, Subtraction & Overflow)

Learning Objectives

By the end of this lecture, students will be able to:

- Understand why computers use **2's Complement** for arithmetic.
 - Perform **signed binary addition**.
 - Perform **signed binary subtraction** using 2's Complement.
 - Detect **overflow** in signed arithmetic.
 - Understand how the **ALU** performs addition and subtraction.
 - Solve exam-style problems involving signed binary arithmetic.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll learn signed binary arithmetic using 2's Complement, including signed addition, signed subtraction, overflow detection, and how the ALU performs both operations using a single adder circuit."

In the previous lecture, we learned about Signed Number Representation, including Unsigned Numbers, Sign-Magnitude, 1's Complement, and 2's Complement.

Today, we'll see how the CPU actually performs arithmetic with signed numbers.

You may think that addition and subtraction require two different circuits.

Surprisingly, modern processors use the same hardware for both operations.

The secret is **2's Complement**.

Instead of building a separate subtraction circuit, the ALU converts subtraction into addition using the 2's Complement of the second number.

In today's lecture, we'll learn this complete process with multiple solved examples and understand how overflow is detected.

Let's begin.

2. Why Use 2's Complement for Arithmetic?

Imagine building a CPU.

If we create:

- One circuit for addition
- Another circuit for subtraction

The processor becomes:

- More expensive
- More complex
- Slower

Instead, processors use:

Subtraction = Addition of the 2's Complement

This allows the **same adder circuit** to perform both operations.

3. Rule for Signed Addition

When adding two signed binary numbers:

1. Add the binary numbers normally.
 2. Ignore the carry out of the most significant bit (if any).
 3. Check for overflow.
-

4. Example 1 – Positive + Positive

Add:

+5 + +3

Step 1: Convert to 8-bit Binary

+5 = 00000101

+3 = 00000011

Step 2: Add

```
00000101
+ 00000011
-----
00001000
```

Answer:

$$00001000 = +8$$

5. Example 2 – Negative + Positive

Calculate:

$$-5 + +3$$

Step 1

Represent both numbers.

$$+5$$

$$00000101$$

2's Complement of +5:

$$11111011$$

So:

$$-5 = 11111011$$

Positive 3:

$$00000011$$

Step 2

Add:

11111011
+ 00000011

1 11111110

Ignore the carry.

Result:

11111110

Step 3

Convert back to Decimal.

Take 2's Complement:

00000010

Result:

-2

Verification:

-5 + 3 = -2

Correct.

6. Example 3 – Negative + Negative

Calculate:

$$-6 + -7$$

Step 1

$$+6$$

00000110

2's Complement:

11111010

$$+7$$

00000111

2's Complement:

11111001

Step 2

Add:

$$\begin{array}{r} 11111010 \\ + 11111001 \\ \hline 1\ 11110011 \end{array}$$

Ignore carry.

Result:

11110011

Take 2's Complement:

00001101

Answer:

-13

Verification:

-6 + -7 = -13

7. Binary Subtraction Using 2's Complement

Instead of subtracting directly:

$$A - B$$

The CPU performs:

$$A + (2\text{'s Complement of } B)$$

This is why the ALU only needs an **adder**.

8. Example 1 – Positive - Positive

Calculate:

$$8 - 5$$

Step 1

8

00001000

5

00000101

Step 2

Take 2's Complement of 5.

11111011

Step 3

Add:

00001000

+ 11111011

1 00000011

Ignore carry.

Answer:

3

Correct.

9. Example 2 – Positive - Negative

Calculate:

8 - (-3)

Remember:

Subtracting a negative

=

Adding a positive

8 + 3

Binary:

00001000

+

00000011

Result:

00001011

Answer:

11

10. Example 3 – Negative - Positive

Calculate:

$$-5 - 3$$

Equivalent to:

$$-5 + (-3)$$

Already represented as:

11111011

+

11111101

Add:

11111000

Convert back:

Result:

-8

11. Overflow

Overflow occurs when the result cannot be represented within the available number of bits.

Overflow Rule

Overflow occurs when:

- Two positive numbers produce a negative result.
 - Two negative numbers produce a positive result.
-

12. Example of Overflow

Suppose we use **4-bit signed numbers**.

Maximum:

+7

Minimum:

-8

Now calculate:

7 + 3

Binary:

0111

+

0011

1010

1010 represents a negative number in 4-bit 2's Complement.

The correct decimal answer is **10**, but 10 cannot be represented using only 4 bits.

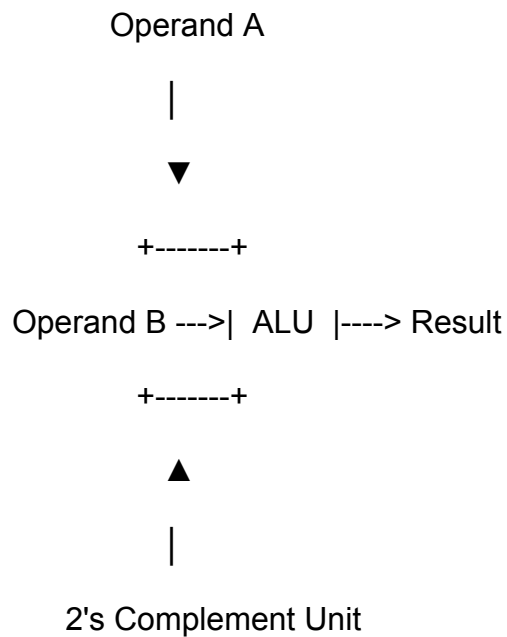
Therefore:

Overflow occurred.

13. Overflow Detection Rules

Operand 1	Operand 2	Result	Overflow
Positive	Positive	Negative	✓ Yes
Negative	Negative	Positive	✓ Yes
Positive	Negative	Either	✗ No

14. How the ALU Performs Addition and Subtraction



For subtraction:

Operand B

↓

2's Complement

↓

Adder



Result

The same hardware performs both addition and subtraction.

15. Real-Life Analogy

Imagine walking on a number line.

- Moving right means **adding**.
- Moving left means **subtracting**.

Instead of creating a new road for moving left, you simply turn around and walk in the opposite direction.

Similarly, the CPU converts subtraction into addition by using the **2's Complement**.

16. Common Mistakes

Students often:

- ✗ Forget to ignore the final carry.
 - ✓ Ignore the carry out of the MSB in standard 2's Complement arithmetic.
-

- ✗ Forget to convert the result back to Decimal.

✓ If the MSB is **1**, take the 2's Complement to find the magnitude and attach a negative sign.

✗ Assume every carry means overflow.

✓ Carry and overflow are **not the same**.

Overflow depends on the **signs** of the operands and the result.

Lecture Summary

Today, we learned how processors perform signed arithmetic using 2's Complement.

We saw that subtraction can be converted into addition by taking the 2's Complement of the second operand, allowing the ALU to use a single adder circuit for both operations. We also learned how to detect overflow and practiced solving signed addition and subtraction problems.

These concepts are fundamental to understanding processor arithmetic, compiler-generated machine code, and Assembly Language instructions.

Thank you for watching, and I'll see you in the next lecture.